AVINASH PAL • LINUX SYSTEMS ENGINEERING

Think Like a Linux Network Engineer

The Linux Network Stack: sk_buff, TCP/IP, Netfilter, eBPF & High-Speed Drivers

TOPIC: NETWORKING & PROTOCOLS

TABLE OF CONTENTS (FULL EBOOK SYLLABUS)

1. The Linux Network Subsystem Architecture
2. NAPI, Ring Buffers, and NIC Driver Interrupts
3. Socket Buffer (sk_buff) Anatomy & Memory Layout
4. TCP/IP Protocol Processing Paths in the Kernel
5. Netfilter, Conntrack, and Routing Table Engines
6. eBPF, XDP (eXpress Data Path), and Kernel Bypass
7. Network Performance Tuning and Latency Optimization

CHAPTER 1 EXCERPT & TECHNICAL NOTES

Author: Avinash Pal | Target: Linux 6.x LTS

1. Packet Flow from Wire to Userspace

1. Packet arrives at physical NIC PHY -> DMA written into RX ring buffer in host memory.
2. Hard IRQ fires -> Driver schedules NAPI poll via netif_rx() or napi_schedule().
3. NAPI loop allocates or wraps sk_buff, passes to __netif_receive_skb().
4. TCP/IP stack decodes headers, passes through Netfilter PREROUTING hooks, and queues to socket.

2. High Performance Tuning with ethtool & sysctl

• ethtool -G eth0 rx 4096 tx 4096 # Maximize hardware ring buffer depths
• sysctl -w net.core.rmem_max=16777216 # 16MB max TCP receive buffer size
• sysctl -w net.ipv4.tcp_congestion_control=bbr # Enable Google BBR algorithm
• ip link set dev eth0 xdpgeneric obj xdp_filter.o sec xdp # Attach eBPF XDP filter

3. sk_buff Headroom and Tailroom

sk_buff uses head, data, tail, and end pointers to allow prepending protocol headers (skb_push) and appending payload data (skb_put) without re-allocating memory buffers.